We can do better

Observed latency: 3.55s.

We can do better

Observed latency: 3.55s.
A SGEMM with $m, n, k = 8192$ requires $2(8192)^3 = 1.10 \times 10^{12}$ FLOPs.

We can do better

Observed latency: 3.55s.
A SGEMM with $m, n, k = 8192$ requires $2(8192)^3 = 1.10 \times 10^{12}$ FLOPs.
These benchmarks were run on an RTX 3090 with **35.6 TFLOP/s** of peak FP32 **throughput**.

We can do better

Observed latency: 3.55s.
A SGEMM with $m, n, k = 8192$ requires $2(8192)^3 = 1.10 \times 10^{12}$ FLOPs.
These benchmarks were run on an RTX 3090 with **35.6 TFLOP/s** of peak FP32 **throughput**.
Therefore, if we achieve peak FP32 throughput, we should have a latency of

$$\frac{1.10 \text{ TFLOP}}{35.6 \text{ TFLOP/s}} \approx 0.031\text{s} = 31\text{ms}.$$

We can do better

Observed latency: 3.55s.
A SGEMM with $m, n, k = 8192$ requires $2(8192)^3 = 1.10 \times 10^{12}$ FLOPs.
These benchmarks were run on an RTX 3090 with **35.6 TFLOP/s** of peak FP32 **throughput**.
Therefore, if we achieve peak FP32 throughput, we should have a latency of

$$\frac{1.10 \text{ TFLOP}}{35.6 \text{ TFLOP/s}} \approx 0.031\text{s} = 31\text{ms}.$$

Whats the hold up?

Our observed latency of 3.55 *seconds* is more than 100× the theoretical minimum of 31 *milliseconds*!

We aren't considering memory bandwidth.

Every thread is **reading everything from DRAM** for each element it needs.

We aren't considering memory bandwidth.

Every thread is **reading everything from DRAM** for each element it needs.
We assigned a thread to each output element, so we have $m \times n \times (2k) = 2(8192)^3$ DRAM accesses for the matmul portion. At FP32, that is $2(8192)^3 \times 4$ bytes ≈ 3298.5 GB.

We aren't considering memory bandwidth.

Every thread is **reading everything from DRAM** for each element it needs.
We assigned a thread to each output element, so we have $m \times n \times (2k) = 2(8192)^3$ DRAM accesses for the matmul portion. At FP32, that is $2(8192)^3 \times 4$ bytes ≈ 3298.5 GB.
The RTX 3090 has a DRAM bandwidth of 935.8 GB/s. So the time required to serve all the DRAM requests is $3298.5/935.8 \approx 3.52$s.

We aren't considering memory bandwidth.

Every thread is **reading everything from DRAM** for each element it needs.
We assigned a thread to each output element, so we have $m \times n \times (2k) = 2(8192)^3$ DRAM accesses for the matmul portion. At FP32, that is $2(8192)^3 \times 4$ bytes ≈ 3298.5 GB.
The RTX 3090 has a DRAM bandwidth of 935.8 GB/s. So the time required to serve all the DRAM requests is $3298.5/935.8 \approx 3.52$s.

We are memory bound!

Our observed latency of 3.55 seconds is dominated by the time it takes to move the data from DRAM to the CUDA Cores, so we are **memory bound**. Our CUDA Cores are sitting idle for most of the time!

Shared Memory (SMEM) Caching

Programmable L1 Cache (SMEM)

Remember how our launch config. required us to group threads as blocks? That is because **blocks** are scheduled to SM's as a unit, so threads in the same block have access to the SM's L1 cache/SMEM.

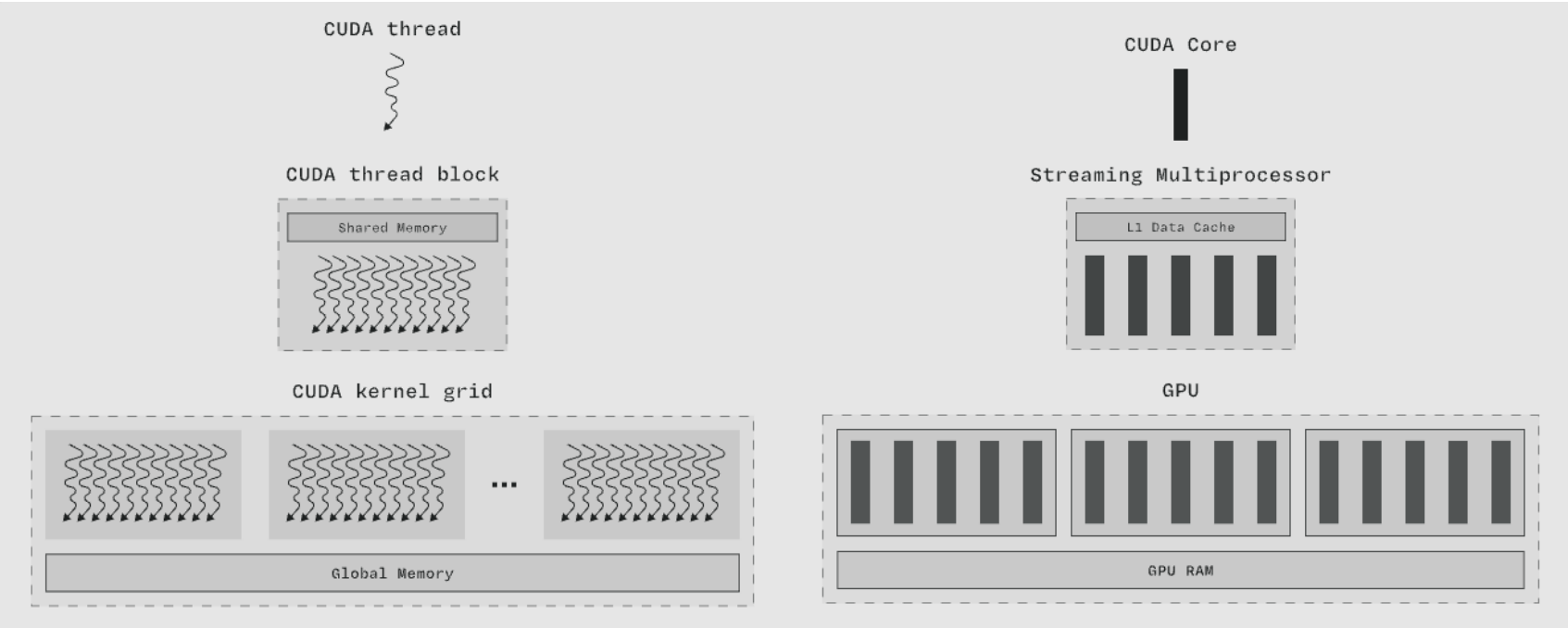


Image source: Modal, "CUDA Programming Model," *GPU Glossary*.

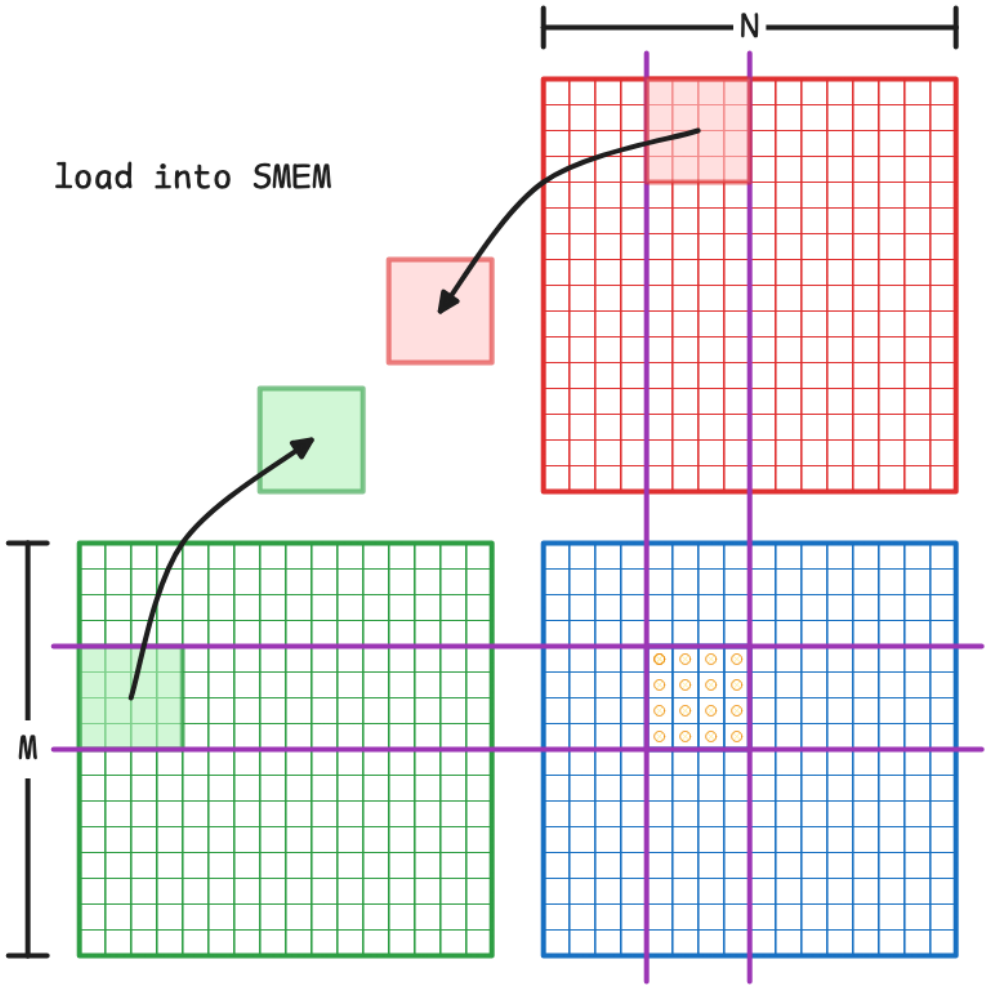
Tiling

Threads can save time by loading data from Device/Global memory into Shared Memory within the SM.

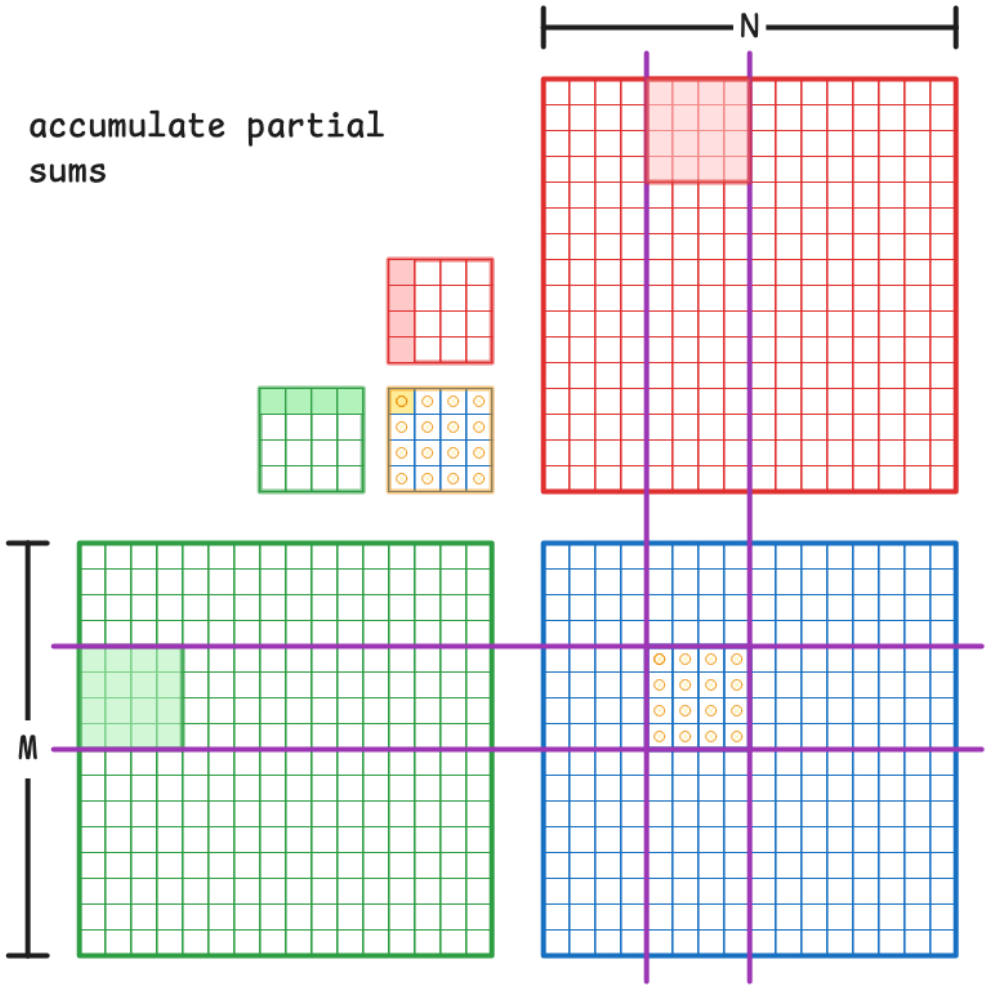
Main Idea

Before doing any computation, load a chunk of the necessary data from DRAM to SMEM as a block. Then, memory accesses for the computation now only go to SMEM.

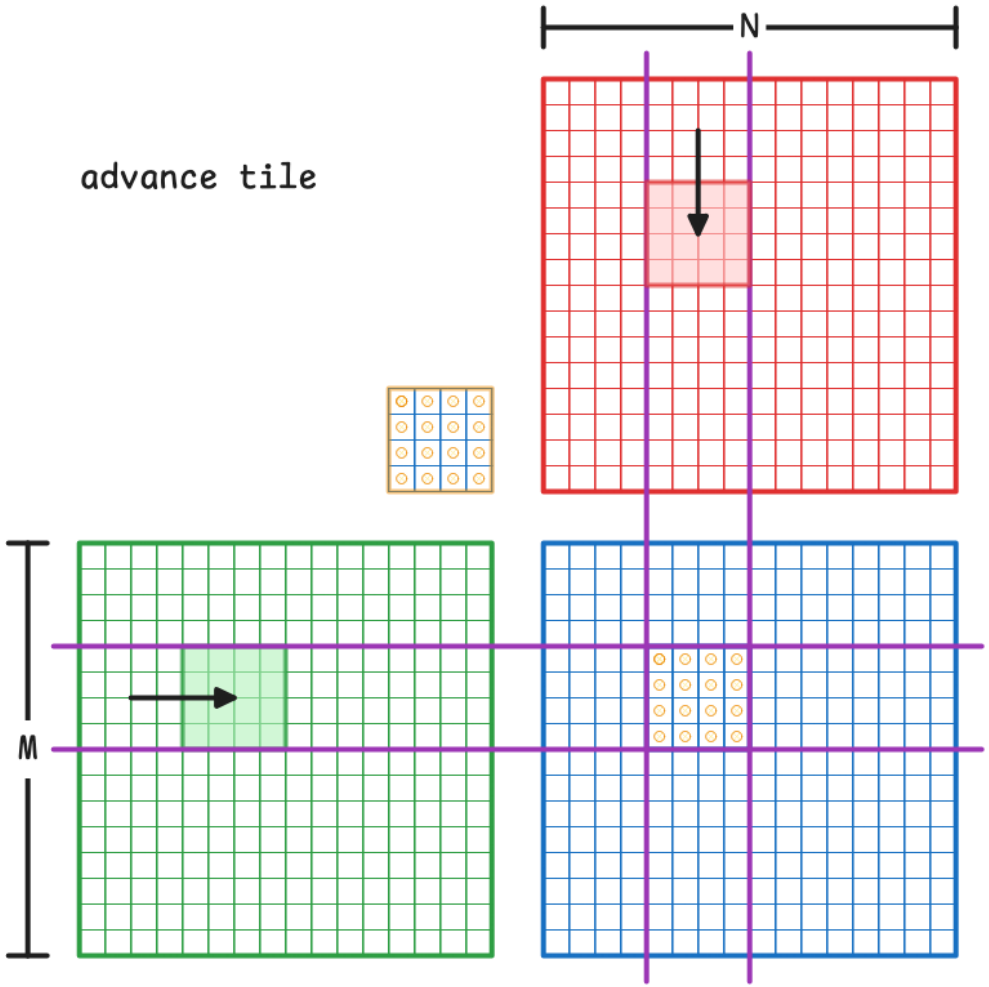
Tiling steps



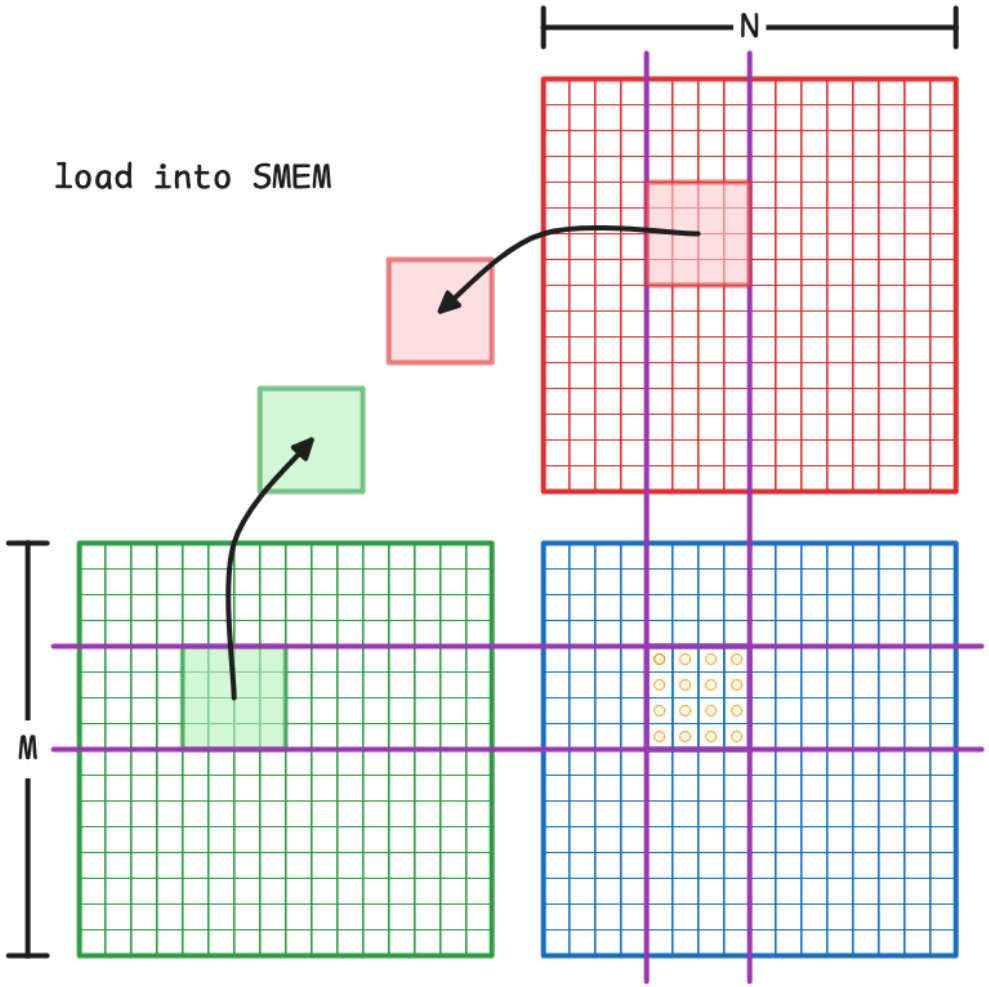
Tiling steps



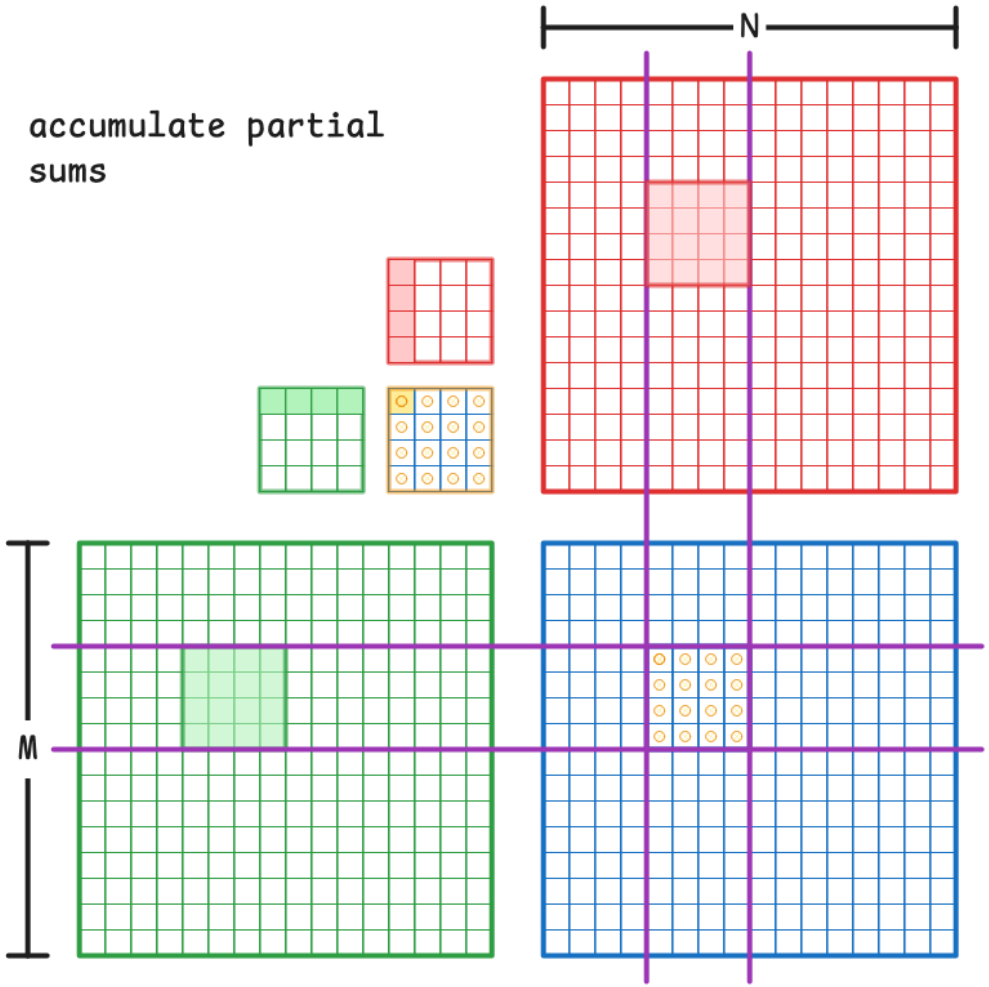
Tiling steps



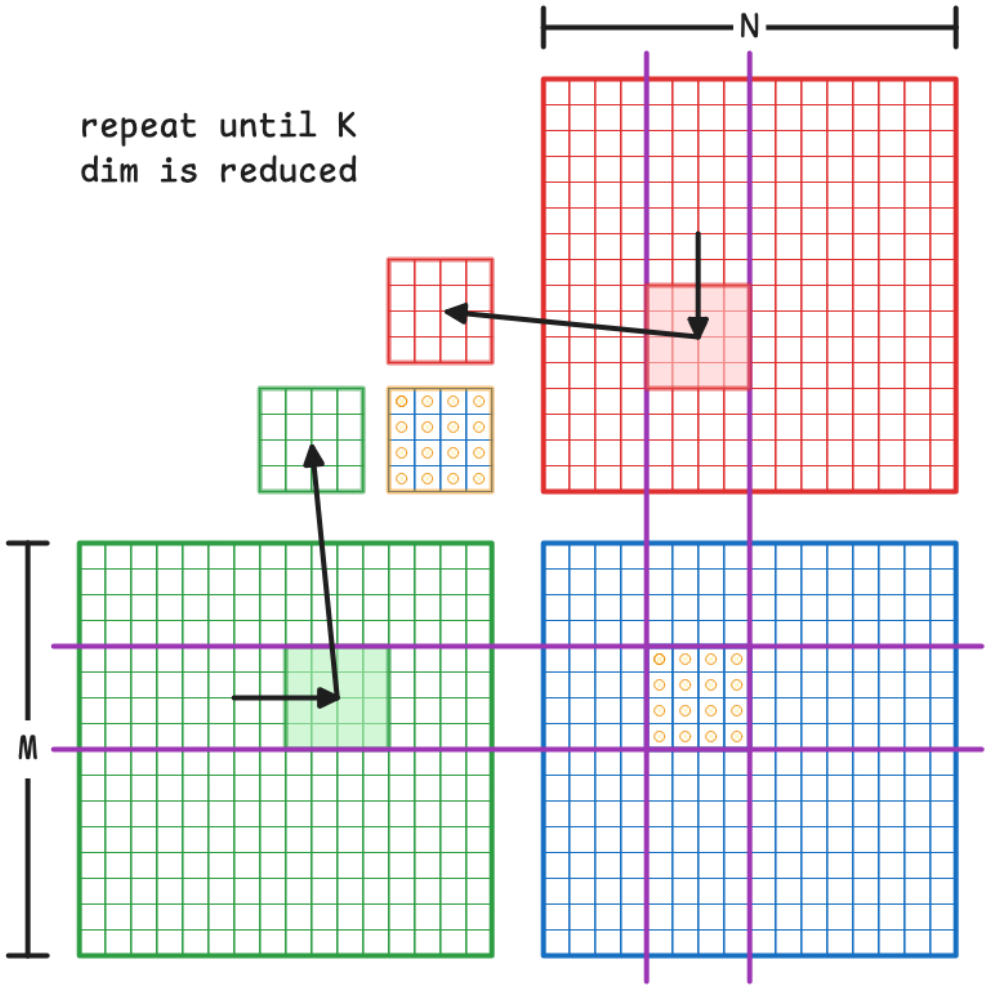
Tiling steps



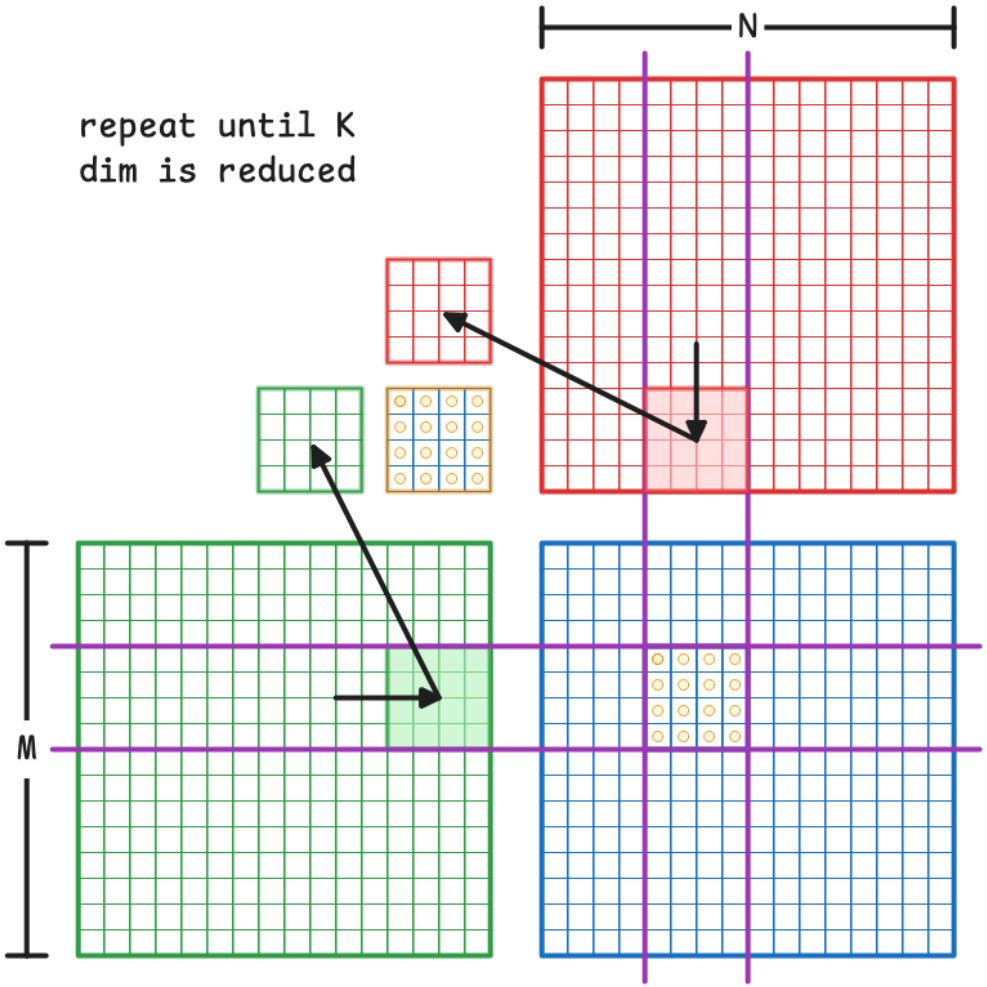
Tiling steps



Tiling steps



Tiling steps



Basic tiled SGEMM

```
__shared__ float As[BM * BK];
__shared__ float Bs[BK * BN];
float tmp = 0.0;

for (int bkIdx = 0; bkIdx < K; bkIdx += BK) {
  As[threadRow * BK + threadCol] =
      A[threadRow * K + threadCol];
  Bs[threadRow * BN + threadCol] =
      B[threadRow * N + threadCol];

  __syncthreads();
  A += BK;
  B += BK * N;

  for (int dotIdx = 0; dotIdx < BK; ++dotIdx) {
    tmp += As[threadRow * BK + dotIdx] *
           Bs[dotIdx * BN + threadCol];
  }

  __syncthreads();
}
```

Cooperate
Each thread loads one value from A and one from B.

Synchronize
Wait until both shared tiles are complete.

Reuse
One thread accumulates from the cached row and column.

Advance
Protect the cache, move along K, and repeat.

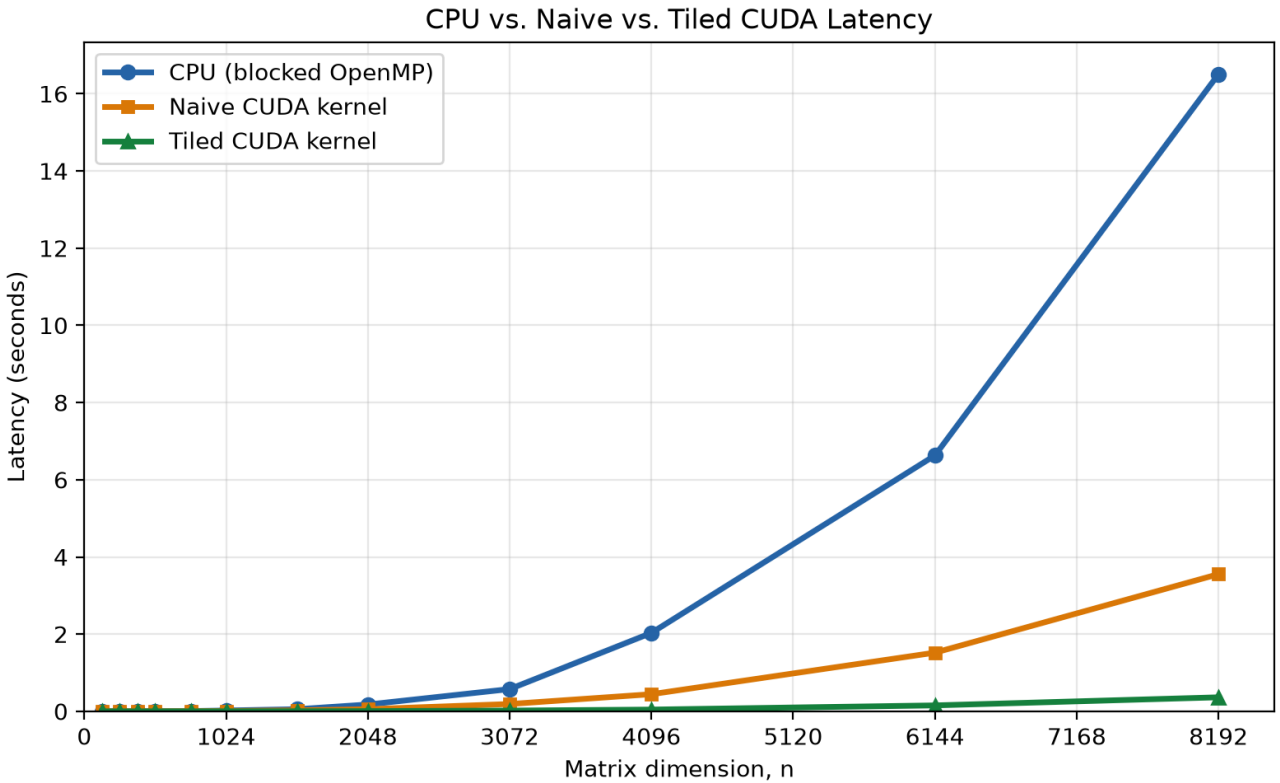
Adapted from Boehm (2022), kernel 3.

Basic tiled SGEMM visualization

- Watch each block load small regions of A and B together.
- The loaded tiles are reused to update an entire tile of C.
- Synchronization separates loading, computing, and advancing to the next tiles.

Play tiled-kernel visualization

Tiling cuts latency by another order of magnitude



Can we still do better?

Throughput analysis

We report a final latency of 0.36s or 360ms for $n = 8192$. This is around $10\times$ the theoretical minimum of 31ms. This slowdown suggests we are still **memory bound**.

Can we still do better?

Throughput analysis

We report a final latency of 0.36s or 360ms for $n = 8192$. This is around $10\times$ the theoretical minimum of 31ms. This slowdown suggests we are still **memory bound**.

This tiling setup with 32×32 blocks and $n, m, k = 8192$, we need a **grid** of $\frac{m}{32} \times \frac{n}{32} = 256 \times 256 = 65,536$ blocks.

Can we still do better?

Throughput analysis

We report a final latency of 0.36s or 360ms for $n = 8192$. This is around $10\times$ the theoretical minimum of 31ms. This slowdown suggests we are still **memory bound**.

This tiling setup with 32×32 blocks and $n, m, k = 8192$, we need a **grid** of $\frac{m}{32} \times \frac{n}{32} = 256 \times 256 = 65,536$ blocks.
Each block loads $\frac{k}{32} = 256$ tiles from both A and B.

Can we still do better?

Throughput analysis

We report a final latency of 0.36s or 360ms for $n = 8192$. This is around $10\times$ the theoretical minimum of 31ms. This slowdown suggests we are still **memory bound**.

This tiling setup with 32×32 blocks and $n, m, k = 8192$, we need a **grid** of $\frac{m}{32} \times \frac{n}{32} = 256 \times 256 = 65,536$ blocks.
Each block loads $\frac{k}{32} = 256$ tiles from both A and B. Each tile is 32×32 elements.

Can we still do better?

Throughput analysis

We report a final latency of 0.36s or 360ms for $n = 8192$. This is around $10\times$ the theoretical minimum of 31ms. This slowdown suggests we are still **memory bound**.

This tiling setup with 32×32 blocks and $n, m, k = 8192$, we need a **grid** of $\frac{m}{32} \times \frac{n}{32} = 256 \times 256 = 65,536$ blocks.
Each block loads $\frac{k}{32} = 256$ tiles from both A and B. Each tile is 32×32 elements.
Thus, we have

$$\underbrace{65,536}_{\text{blocks}} \times \underbrace{256}_{\text{tiles/block}} \times \underbrace{2}_{A \text{ and } B} \times \underbrace{32 \times 32}_{\text{elements/tile}} \times \underbrace{4 \text{ bytes}}_{\text{bytes/element}} \approx 137.7 \text{ GB}.$$

Can we still do better?

Throughput analysis

We report a final latency of 0.36s or 360ms for $n = 8192$. This is around $10\times$ the theoretical minimum of 31ms. This slowdown suggests we are still **memory bound**.

This tiling setup with 32×32 blocks and $n, m, k = 8192$, we need a **grid** of $\frac{m}{32} \times \frac{n}{32} = 256 \times 256 = 65,536$ blocks.
Each block loads $\frac{k}{32} = 256$ tiles from both A and B. Each tile is 32×32 elements.
Thus, we have

$$\underbrace{65,536}_{\text{blocks}} \times \underbrace{256}_{\text{tiles/block}} \times \underbrace{2}_{A \text{ and } B} \times \underbrace{32 \times 32}_{\text{elements/tile}} \times \underbrace{4 \text{ bytes}}_{\text{bytes/element}} \approx 137.7 \text{ GB}.$$

At 935.8 GB/s, we need 0.147s or 147ms to move all data from DRAM to SMEM.

This type of analysis is getting hard...

When we did this analysis for the **naive kernel**, it was easy to see how the data moved, and at the end of it, we even got the nice result of

$$\underbrace{3.55\text{s}}_{\text{Observed latency}} \approx \underbrace{3.52\text{s}}_{\text{Time to move data}} + \underbrace{0.031\text{s}}_{\text{Theoretical minimum for work}} .$$

This type of analysis is getting hard...

When we did this analysis for the **naive kernel**, it was easy to see how the data moved, and at the end of it, we even got the nice result of

$$\underbrace{3.55\text{s}}_{\text{Observed latency}} \approx \underbrace{3.52\text{s}}_{\text{Time to move data}} + \underbrace{0.031\text{s}}_{\text{Theoretical minimum for work}} .$$

However, as we apply more optimizations, **predicting performance based on hardware specifications becomes impractical.**

This type of analysis is getting hard...

When we did this analysis for the **naive kernel**, it was easy to see how the data moved, and at the end of it, we even got the nice result of

$$\underbrace{3.55\text{s}}_{\text{Observed latency}} \approx \underbrace{3.52\text{s}}_{\text{Time to move data}} + \underbrace{0.031\text{s}}_{\text{Theoretical minimum for work}}.$$

However, as we apply more optimizations, **predicting performance based on hardware specifications becomes impractical.** Even with just our **basic tiling kernel**, the observed latency of

$$\underbrace{360\text{ms}}_{\text{Observed latency}} \neq \underbrace{147\text{ms}}_{\text{Time to move data}} + \underbrace{32\text{ms}}_{\text{Theoretical minimum for work}} = 179\text{ms}.$$

This type of analysis is getting hard...

When we did this analysis for the **naive kernel**, it was easy to see how the data moved, and at the end of it, we even got the nice result of

$$\underbrace{3.55\text{s}}_{\text{Observed latency}} \approx \underbrace{3.52\text{s}}_{\text{Time to move data}} + \underbrace{0.031\text{s}}_{\text{Theoretical minimum for work}}.$$

However, as we apply more optimizations, **predicting performance based on hardware specifications becomes impractical.** Even with just our **basic tiling kernel**, the observed latency of

$$\underbrace{360\text{ms}}_{\text{Observed latency}} \neq \underbrace{147\text{ms}}_{\text{Time to move data}} + \underbrace{32\text{ms}}_{\text{Theoretical minimum for work}} = 179\text{ms}.$$

This gap comes from things like occupancy, synchronization overhead, bank conflicts...

This type of analysis is getting hard...

When we did this analysis for the **naive kernel**, it was easy to see how the data moved, and at the end of it, we even got the nice result of

$$\underbrace{3.55\text{s}}_{\text{Observed latency}} \approx \underbrace{3.52\text{s}}_{\text{Time to move data}} + \underbrace{0.031\text{s}}_{\text{Theoretical minimum for work}}.$$

However, as we apply more optimizations, **predicting performance based on hardware specifications becomes impractical.** Even with just our **basic tiling kernel**, the observed latency of

$$\underbrace{360\text{ms}}_{\text{Observed latency}} \neq \underbrace{147\text{ms}}_{\text{Time to move data}} + \underbrace{32\text{ms}}_{\text{Theoretical minimum for work}} = 179\text{ms}.$$

This gap comes from things like occupancy, synchronization overhead, bank conflicts... **Without profiling every possible source of latency, can we still determine whether memory is the bottleneck**?